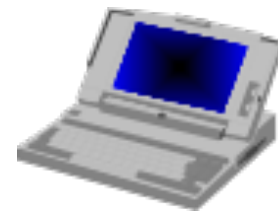


第4章

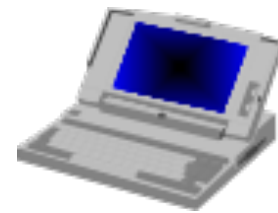
汇编语言程序设计





主要内容

- 汇编语言源程序的结构
- 汇编语言语句格式
- 伪指令
- 功能调用
- 汇编语言程序设计



§4.1 汇编语言源程序

了解：

- 汇编语言源程序的结构
- 汇编语言语句类型及格式



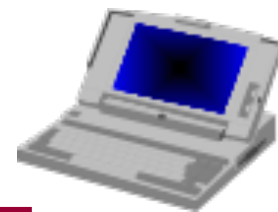
一、汇编语言源程序结构



1. 汇编语言源程序与汇编程序

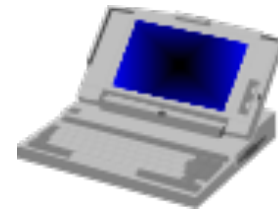
- 汇编语言源程序 → 用助记符编写
- 汇编程序 → 源程序的编译程序
编译器





2. 汇编语言程序设计与执行过程

- 输入汇编语言源程序 (EDIT) → 源文件 .ASM
- 汇编 (MASM) → 目标文件 .OBJ
- 链接 (LINK) → 可执行文件.EXE
- 调试 (TD) → 最终程序



3. 汇编语言源程序结构

数据段名 SEGMENT

...

数据段名 ENDS

附加段名 SEGMENT

...

附加段名 ENDS

堆栈段名 SEGMENT

...

堆栈段名 ENDS

代码段名 SEGMENT

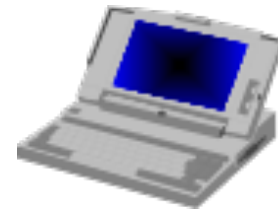
...

代码段名 ENDS

END



二、汇编语言语句类型及格式



1. 汇编语言语句类型

指令性语句



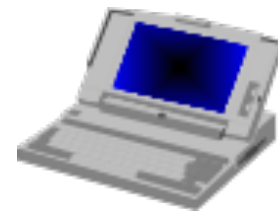
CPU执行的语句，能够生成目标代码

指示性语句



CPU不执行，而由汇编程序执行的语句，不生成目标代码

2. 汇编语言语句格式



指令性语句:

[标号:] [前缀] 助记符 [操作数], [操作数] [; 注释]

指令的符号地址
标号后要有冒号

操作码

注释前加分号

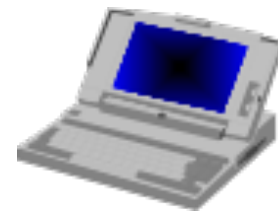
指示性语句:

[名字] 伪指令助记符 操作数 [, 操作数, ...] [; 注释]

变量的符号地址
其后不加冒号

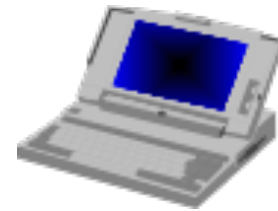
指示性语句中至少有一个操作数

所有操作数相加不超过64K



3. 标号、名字

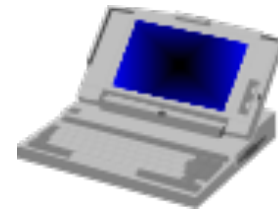
- 标号后有冒号，在指令性语句前
- 名字后不加冒号，在指示性语句前
- 英文字母、数字及专用字符组成,最大长度不能超过31个,且不能由数字打头,不能用保留字(如寄存器名,指令助记符,伪指令)。



4. 助记符

- 操作码 → 命令CPU要完成的操作
- 伪指令助记符
 - 指示汇编程序要完成的操作。
 - 作用：
 - 定义变量
 - 分配存储单元
 - 指示程序开始和结束等

5. 注释



• 用分号作为注释的开始；

• 用于说明程序或语句的功能，以便于阅读和理解

6. 操作数

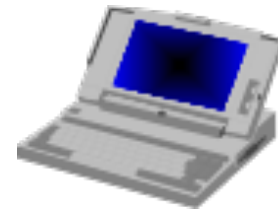
寄存器

存储器单元

常量

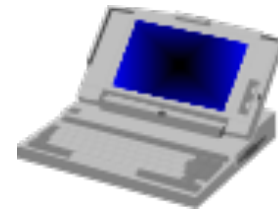
变量或标号的属性

表达式



常量

- 数字常量
- 字符串常量 → 用单引号引起的字符或字符串
- 例：‘A’
- `MOV AL, ‘A’`
- 例：‘ABCD’
→ 汇编时被译成对应的ASCII
码41H, 42H, 43H, 44H



表达式

算术运算

逻辑运算

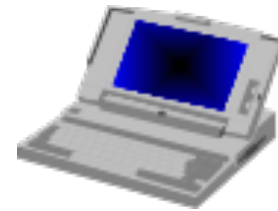
关系运算

取值运算和属性运算

其它运算

算术运算和逻辑运算符

- 算术运算符
 - $+$, $-$, $*$, $/$, MOD
- 逻辑运算符
 - AND, OR, NOT, XOR
- 例:
 - MOV AL, 8 AND 4
 - MOV AL, 8+4-1



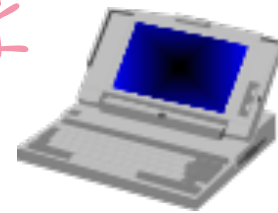
关系运算符

- 表示关系是否成立的运算符
- 参与关系运算的必须是两个数值
- 关系成立，结果=FFFFH（或FFH，用1占各位）
- 关系不成立，结果=0。
- 运算符：
 - EQ, NE, LT, GT, LE, GE

例：MOV AL, 3 LT 4

变量

内存单元的符号地址, 为指令寄存器
作数



- 代表内存中的数据区, 程序中视为存储器操作数

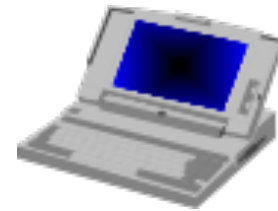
- 变量的属性:

段 值 变量所在段的段地址

偏移量 变量单元地址与段首地址之间的位移量。
偏移地址

类 型 字节型、字型和双字型

取值运算符



- 用于分析存储器操作数的属性

OFFSET → 取得其后变量或标号的偏移地址

SEG → 取得其后变量或标号的段地址

MOV AX, SEG DATA

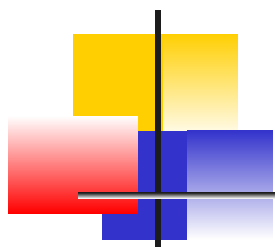
MOV DS, AX

MOV BX, OFFSET DATA

LEA BX, DATA

等价于

取值运算符例



TYPE

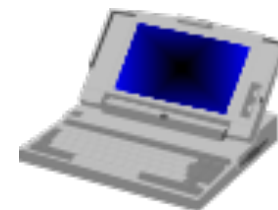
取变量的类型

LENGTH

取所定义存储区的长度

SIZE

取所定义存储区的字节数



- 若BUFFER存储区用如下伪指令定义：

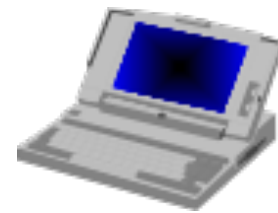
BUFFER DW 200 DUP(0)

则：

TYPE BUFFER 等于2

LENGTH BUFFER 等于200

SIZE BUFFER 等于400



属性运算符

- 用于指定其后存储器操作数的类型

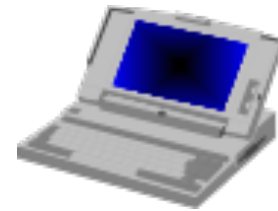
- 运算符: PTR

- 例:

-- WORD --
MOV BYTE PTR[BX], 12H

BX为字型. 0012H

BX为字节型 12H



其它运算符

- 方括号:

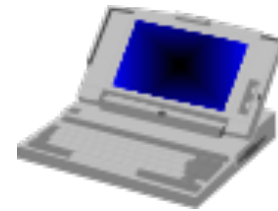
[] → 方括号中内容为操作数的偏移地址

- 段重设符

段寄存器名: [] → 用于修改默认的段基地址

§4.2 伪指令

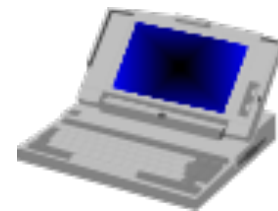
掌握：



- 伪指令的格式及实现的操作
- 伪指令的应用

伪指令

- 由汇编程序执行的“指令系统”
- 作用：
 - 定义变量；
 - 分配存储区
 - 定义逻辑段；
 - 指示程序开始和结束；
 - 定义过程等。



常用伪指令

数据定义伪指令

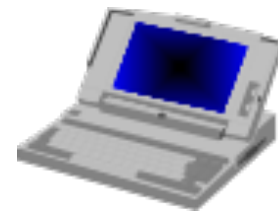
符号定义伪指令

段定义伪指令

结束伪指令

过程定义伪指令

宏命令伪指令



一、数据定义伪指令

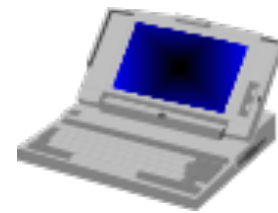
- 用于定义数据区中变量的类型及大小
- 格式：

[变量名] 伪指令助记符 操作数, ... ; [注释]

表示符号地址

定义变量类型

定义变量值
及区域大小



1. 数据定义伪指令助记符

- **DB** 定义的变量为字节型 *Define Byte*
- **DW** 定义的变量为字类型 (双字节) *Define Word*
- **DD** 定义的变量为双字型 (4字节) *Define Double Word*
- **DQ** 定义的变量为4字型 (8字节) *Define Quadword*
- **DT** 定义的变量为10字节型 *Define Ten bytes*



- DATA1

11
22
33
44

DATA2

11
00
22
00
44
33

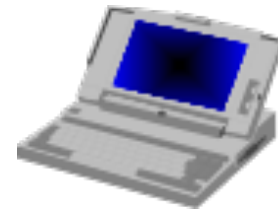
DATA3

22
00
00
00
22
00
00
00
66
55
00
33

00 00 00 22 H

以上变量在内存中的存放形式?

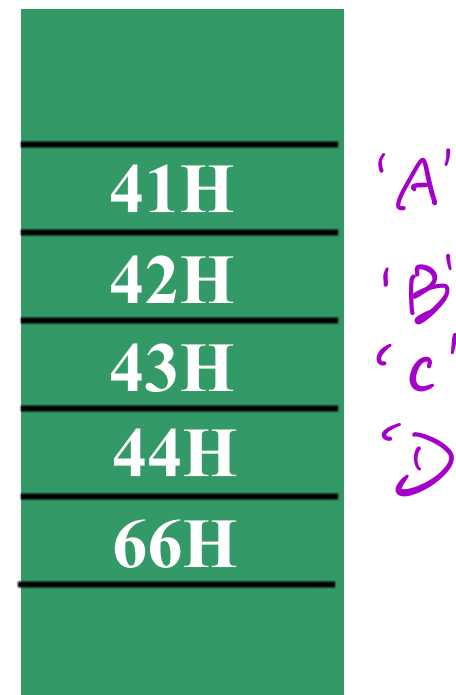
以上变量在内存中的存放形式

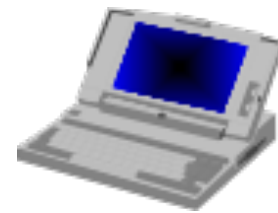


数据定义伪指令的几点说明

- 伪指令的性质决定所定义变量的属性；
- 定义字符串必须用DB伪指令
- 例：

DATA1 DB 'ABCD', 66H





2. 重复操作符

- 作用：

- 为一个数据区的各单元设置**相同**的初值

- 目的：

- 常用于声明一个数据区

- 格式：

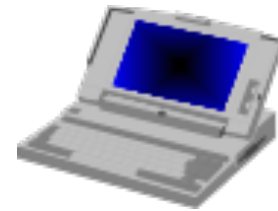
[变量名] 伪指令助记符 **(n)** DUP (初值, ...)

- 例：

DATA BW 20 DUP (0)

重复次数

重复内容



3. “?”的作用

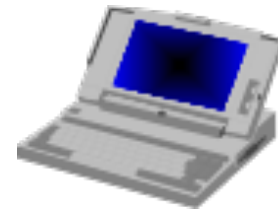
- 表示随机值，用于预留存储空间
- `MEM1 DB 34H, 'A', ?`

`MEM2 DW 20 DUP ( ? )`

20个单元

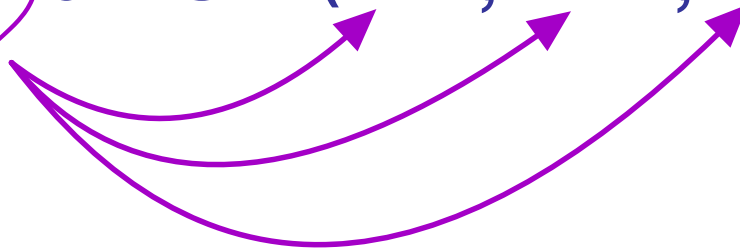


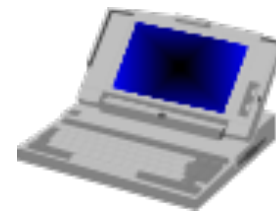
预留40个字节单元，每单元为随机值



数据定义伪指令例

- M1 DB 'How are you?'
- M2 DW 3 DUP(^{0011H}11H), 3344H
- DB 4 DUP (?)
- M3 DB 3 DUP (22H, 11H, ?)





数据定义伪指令例

M1

'H'
'o'
'w'
' '
'a'
'r'
'e'
' '
'y'
'o'
'u'
'?'

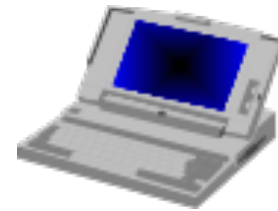
M2

11H
00H
11H
00H
11H
00H
44H
33H
XX
XX
XX
XX

M3

22H
11H
XX
22H
11H
XX
22H
11H
XX

随机数



二、符号定义伪指令

- 格式：

符号名 EQU 表达式

- 操作：

用符号名取代后边的表达式，不可重新定义

- 例：

CONSTANT EQU 100

VAR EQU 30H+99H

EQU说明的表达式不占用内存空间

三、段定义伪指令



- 说明逻辑段的起始和结束；

- 说明不同程序模块中同类逻辑段之间的联系形态

段名 SEGMENT [定位类型] [组合类型] ['类别']

段名 ENDS

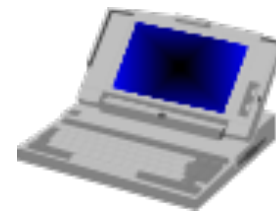
说明逻辑段的起点

说明不同模块中同名段的组合连接方式

链接时不同程序模块中同类逻辑段将被装入连续存储空间区域

逻辑段的起始地址

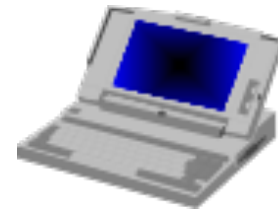
段名 SEGMENT [定位类型] [组合类型] [“类别”]



定位类型

- **PARA:** 段的起点从节边界开始
(16个字节为1节)
- **BYTE:** 段的起点从存储器任何地址开始
- **WORD:** 段的起点从偶地址开始
- **PAGE:** 段的起点从页边界开始
(256个字节为1页)

段名 SEGMENT [定位类型] [组合类型] [“类别”]



组合类型

- 与其它模块中的同名段在满足定位类型的前提下具有的组合方式：
 - NONE: 不组合
 - PUBLIC: 依次连接（顺序由LINK程序确定）
 - COMMON: 覆盖连接
 - STACK: 堆栈段的依次连接
 - AT 表达式: 段定义在表达式值为段基的节边界
 - MEMORY: 相应段在同名段的最高地址处。

四、设定段寄存器伪指令



说明所定义逻辑段的性质

格式：

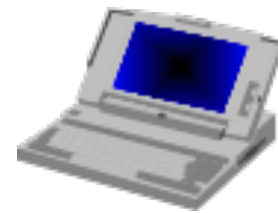
ASSUME 段寄存器名：段名
[, 段寄存器名：段名, ...]

五、结束伪指令

- 表示源程序结束

- 格式：

END [标号]



一个完整源程序结构例

DSEG SEGMENT

DATA1 DB 1, 2,

DATA2 DW 1234H

DSEG ENDS

ESEG SEGMENT

DB 20 DUP (?)

ESEG ENDS

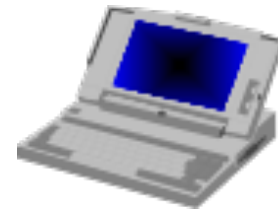
SSEG SEGMENT STACK 'STACK'

DB 200 DUP (?)

SSEG ENDS

通令变量

栈



一个完整源程序结构例

CSEG SEGMENT

**ASSUME CS: CSEG, DS: DSEG,
ES: ESEG, SS: SSEG**

**START: MOV AX, DSEG
MOV DS, AX
MOV AX, ESEG
MOV ES, AX
MOV AX, SSEG
MOV SS, AX**

CSEG ENDS

END START

段寄存器
初始化
——将
段地址
送相应

源程序代码

数据段名 SEGMENT

...

数据段名 ENDS

附加段名 SEGMENT

...

附加段名 ENDS

堆栈段名 SEGMENT

...

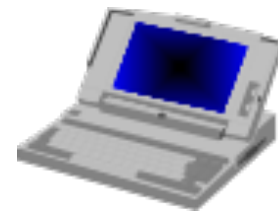
堆栈段名 ENDS

代码段名 SEGMENT

...

代码段名 ENDS

END



六、过程定义伪指令

- 用于定义一个过程体
- 格式：

过程名 **PROC** **[NEAR / FAR]**

⋮

RET

过程名 **ENDP**

若为近过程，NEAR可省略

过程体最后一条指令是RET

过程定义及调用例

■ 定义延时子程序

DELAY PROC

PUSH BX

PUSH CX

MOV BL, 2

NEXT: MOV CX, 4167

W10M: LOOP W10M

DEC BL

JNZ NEXT

POP CX

POP BX

RET

DELAY ENDP

■ 调用延时子程序：

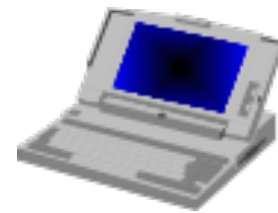
CALL DELAY

成环

} 参数保护

2x 4167 次

} 恢复



七、宏命令伪指令

- 宏 → 源程序中由汇编程序识别的具有独立功能的一段程序代码

- 格式：

宏命令名 **MACRO** <形式参数>

⋮

ENDM

宏体

无宏 ☆ 范围区别

不能通过编译链接
独立运行，只能作为源
程序的一部分

P163例

宏定义与宏调用例

——对应

■ 定义宏：

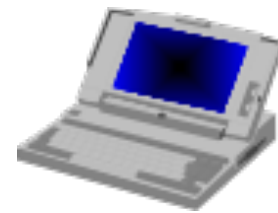
```
DADD MACRO X, Y, Z
    MOV AX, X
    ADD AX, Y
    MOV Z, AX
ENDM
```

■ 宏调用：

```
■ DADD DATA1, DATA2, SUM
```

■ 汇编后源程序中的宏展开：

```
■ MOV AX, DATA1
■ ADD AX, DATA2
■ MOV SUM, AX
```



八、其它伪指令

ORG —— 指明其后程序代码或变量在所在段的起始地址偏移量

- 格式：

ORG 表达式

- 例：

ORG 2000H

计算值为
非负常数

DATA SEGMENT
ORG 200H

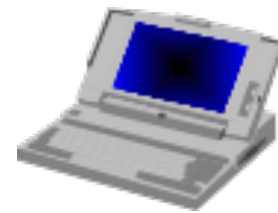
BUFF DB 1,2

DATA ENDS

变量 BUFF 的偏移地址为 120H

§4.3 功能调用

DOS 功能调用



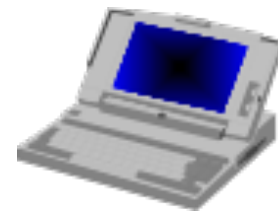
BIOS功能调用

DOS 功能调用

- 包含多个子功能的功能包，用**软中断指**令调用，**中断类型码**固定为**21H**；
- 各子功能采用**功能号**来区分。

驻留在ROM中的基本输入/输出系统
BIOS 功能：加电自检、引导、主要I/O设备驱动程序及接口控制

附录D



DOS 功能调用

设备管理
目录管理
文件管理
其它

- 调用格式:

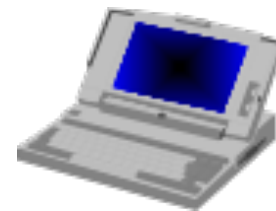
MOV AH, 功能号

<置相应参数>

INT 21H

↓
设置出入口参数

一、单字符输入



调用方法:

```
MOV AH, 01
```

```
INT 21H
```

输入的字符在AL中

```
GET_KEY: MOV AH,1
```

```
INT 21H
```

```
CMP AL,'Y'
```

```
JZ YES
```

```
CMP AL,'N'
```

```
JZ NO
```

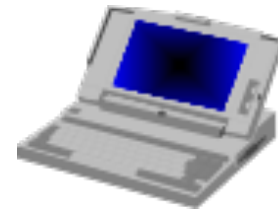
```
JNZ GET_KEY
```

```
YES: |
```

```
NO: |
```

交互式应
答程序

二、字符串输入



调用格式

字符串输入缓冲区的定义

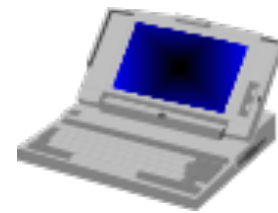
1. 调用格式

• AH ← 功能号 0AH

• DS: DX ← 字符串在内存中的存放地址

• INT 21H

定义在数据段



2. 定义字符缓冲区

- 用户自定义缓冲区格式：

存放字符个数：≤65533



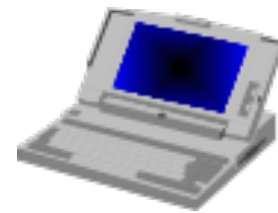
存放键入的字符

回车

整个缓冲区

实际键入字符数

最大可键入字符数



输入字符串程序段

```
DAT1 DB 20, ? , 20 DUP ( ? )
```

```
⋮
```

```
LEA DX, DAT1
```

```
MOV AH, 0AH
```

```
INT 21H
```

在数据段
中定义

接收字符串

缓冲区定义20字节，最多输入19个字符后回车

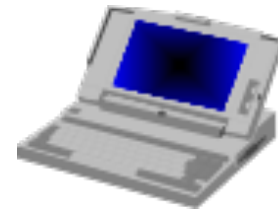
14H

?

0DH

20个字节

三、单字符显示输出



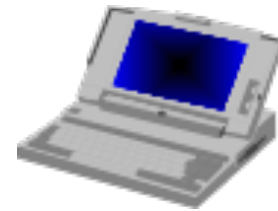
• AH ← 功能号02H

• DL ← 待输出字符

• INT 21H

- MOV AH, 02
- MOV DL, 41H
- INT 21H

执行结果：
屏幕显示A



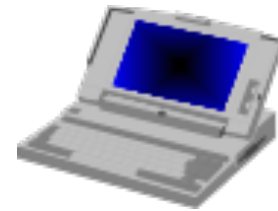
四、字符串输出显示

- AH ← 功能号09H

- DS: DX ← 待输出字符串的偏移地址

- INT 21H

- 被显示的字符串必须以‘\$’结束,且所显示的内容不应出现非可见的ASCII码



字符串输出显示例

DATA SEGMENT

MESS1 DB 'Input String:' 0DH, 0AH, '\$'

DATA ENDS

CODE SEGMENT

⋮

MOV AH, 09

MOV DX, OFFSET MESS1

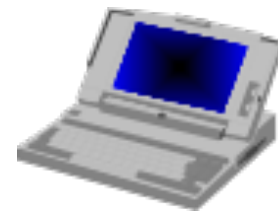
INT 21H

⋮

回车符

换行符

的 ASCII 码



五、返回到DOS

- AH ← 功能号4CH

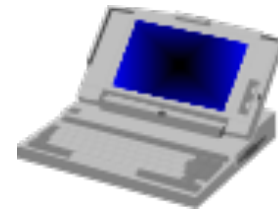
- INT 21H

MOV AH, 4CH

INT 21H

用于结尾

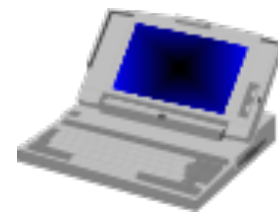
- HLT指令--暂停CPU运行，但没有返回到DOS



§4.4 汇编语言程序设计

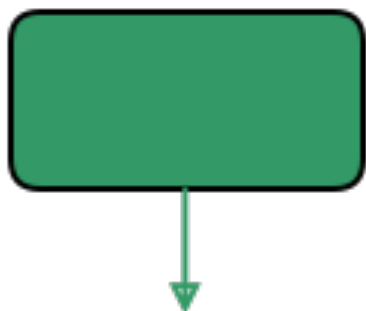
设计步骤:

- 根据实际问题抽象出数学模型
- 确定算法
- 画程序流程图
- 分配内存工作单元和寄存器
- 程序编码
- 调试

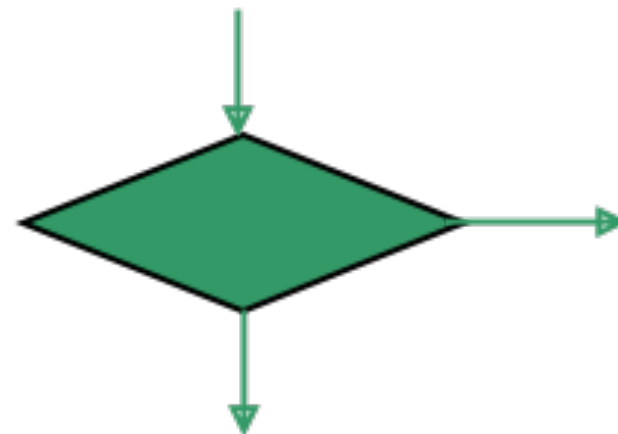


程序流程图符号

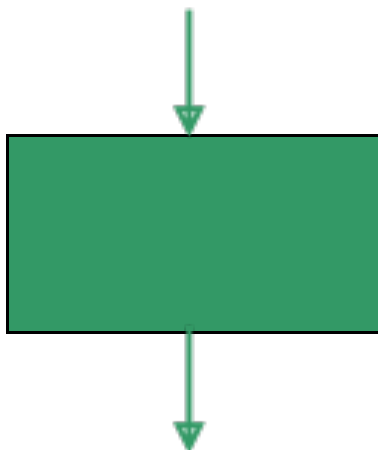
起始：



判断：

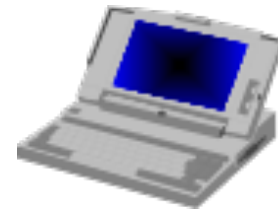


功能：



子过程：





汇编语言程序结构

- 顺序结构
- 循环结构
 - 先判定型
 - 后判定型
- 分枝结构
 - 单分枝
 - 多分枝
- 子程序结构

顺序结构



【例 4-4】求两个数的平均值。这两个数分别放在 x 单元和 y 单元中,而平均值放在 z 单元中。源程序如下:

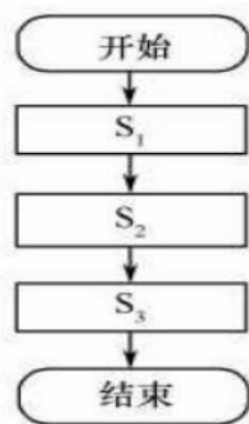


图 4-5 顺序结构流程图

```
DATA    SEGMENT
        x DB 95
        y DB 87
        z DB ?
DATA    ENDS
```

```
CODE    SEGMENT
ASSUME CS : CODE, DS : DATA
START:  MOV AX, DATA      ;装填数据段寄存器 DS
        MOV DS, AX
        MOV AL, x         ;第一个数送入 AL
        ADD AL, y         ;两数相加,结果送 AL
        MOV AH, 0
        ADC AH, 0         ;带进位加法,进位送 AH
        MOV BL, 2         ;除数 2 送 BL
        DIV BL            ;求平均值送 AL
        MOV z, AL        ;结果送入 z 单元
        MOV AH, 4CH
        INT 21H
CODE    ENDS
END START
```

分支结构



给定以下符号函数：

$$y = \begin{cases} 1 & x > 0 \\ 0 & x = 0 \\ -1 & x < 0 \end{cases}$$

任意给定 x 值,存放在 x 单元,函数值 y 存放在 y 单元,根据 x 的值确定函数 y 的值。

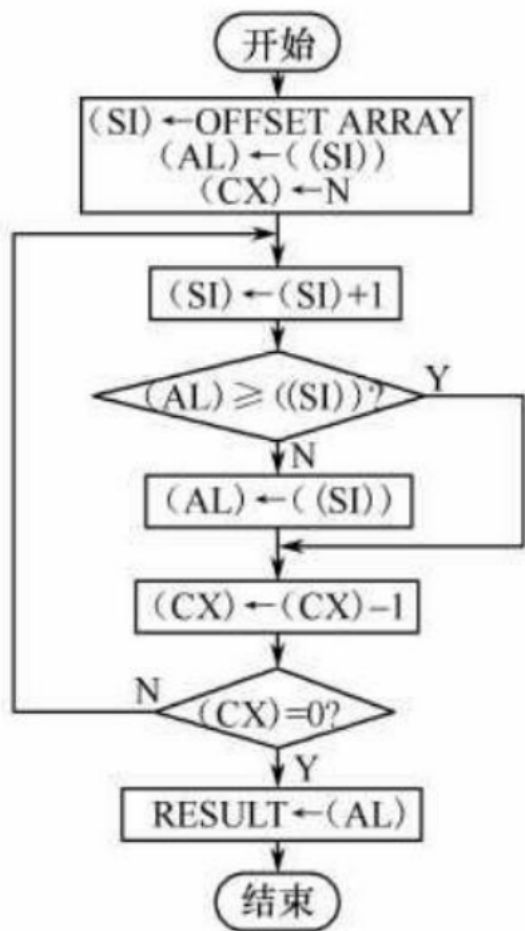
<pre> DATA SEGMENT x DB 36 y DB ? DATA ENDS CODE SEGMENT MAIN PROC FAR ASSUME CS : CODE, DS : DATA START: PUSH DS SUB AX,AX PUSH AX MOV AX,DATA MOV DS,AX </pre>	<pre> MOV AL,x ;AL←x CMP AL,0 JGE LOOP1 ;x≥0 时转 LOOP1 MOV y,0FFH ;y←-1 RET LOOP1: JE LOOP2 ;x=0 时转 LOOP2 MOV y,1 ;y←1 RET LOOP2: MOV y,0 ;y←0 RET MAIN ENDP CODE ENDS END START </pre>	<pre> MOV AH,4CH INT 21H </pre>
--	---	---

循环结构



【例 4-10】从 ARRAY 单元开始的 N 个连续单元中存放有 N 个无符号数, 从中找出最大者送入 RESULT 单元中。

根据题意, 把 ARRAY 单元第一个数先送入 AL 寄存器, 将 AL 中的数与后面的 $N-1$ 个数逐个进行比较。如果 AL 中的数较小, 则两数交换位置; 如果 AL 中的数大于等于相比较的数, 则两数不交换位置。在比较过程中, AL 中始终保持较大的数, 比较 $N-1$ 次, 则最大者必在 AL 中。最后把 AL 中的数(最大者)送入 RESULT 单元。程序流程图如图 4-12 所示。



```
DATA    SEGMENT
        ARRAY DB N DUP(?)
        RESULT DB ?
DATA    ENDS
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA
MAIN    PROC FAR
START:  PUSH DS
        SUB AX,AX
        PUSH AX
        MOV AX,DATA
        MOV DS,AX
```

```
        LEA SI,ARRAY
        MOV AL,[SI]
        MOV CX,LENGTH ARRAY
LOOP1:  INC SI
        CMP AL,[SI]
        JAE LOOP2
        XCHG AL,[SI]
LOOP2:  LOOP LOOP1
        MOV RESULT,AL
        RET
MAIN    ENDP
CODE    ENDS
        END START
```


子程序结构

过程名 PROC [NEAR/FAR]
.....
RET

编写一程序,完成对 ARRAY 数组中 N 个元素求和。

过程名 ENDP

```
DATA SEGMENT
    ARRAY DW 30 DUP (?)
    COUNT DW 30
    SUM DW 2 DUP (?)
DATA ENDS

CODE SEGMENT
MAIN PROC FAR
    ASSUME CS:CODE,DS:DATA
START: PUSH DS
        SUB AX,AX
        PUSH AX
        MOV AX,DATA
        MOV DS,AX
        CALL PADD
        RET
MAIN ENDP

PADD PROC NEAR
    PUSH AX
    PUSH CX
    PUSH DX
    PUSH SI
    LEA SI,ARRAY
    MOV CX,COUNT
    XOR AX,AX
    MOV DX,AX
AGAIN: ADD AX,[SI]
        JNC NEXT
        INC DX
NEXT:  INC SI
        INC SI
    LOOP AGAIN
    LEA SI,SUM
    MOV [SI],BX
    MOV [SI+2],DX
    POP SI
    POP DX
    POP CX
    POP AX
    RET
PADD ENDP
CODE ENDS
END START
```

示例1：

变量在内存中的存储
(16进制表示)

以下数据区在内存中是如何存储的？

```
DATA SEGMENT
```

```
NAMES DB 'TOM..', 20
```

```
        DB 'CATE', 25
```

```
DATA ENDS
```

NAMES

54	T	
4F	O	
4D	M	
2E	.	
2E	.	
14	20	
43	C	
41	A	
54	T	
45	E	
19	25	

示例2：

阅读程序段，说明该程序段的功能

```
DATA SEGMENT
```

```
    A DB '123ABC'
```

```
DATA ENDS
```

```
CODE SEGMENT
```

```
ASSUME CS: CODE, DS: DATA
```

```
START: MOV AX, DATA
```

```
        MOV DS, AX
```

```
        LEA BX, A
```

```
        MOV CX, 6
```

```
LP:     MOV AH, 2
```

```
        MOV AL, [BX]
```

```
        XCHG AL, DL
```

```
        INC BX
```

```
        INT 21H
```

```
        LOOP LP
```

```
        MOV AH, 4CH
```

```
        INT 21H
```

```
CODE ENDS
```

```
END    START
```

除代码段，
其他段的段
地址的
初始化

循环 CX 次

示例3：

DATA SEGMENT

TABLE DW 3400H, 5600H, 2300H, 4500H, 2300H, 1200H, 2344H, 3500H

DATA ENDS

CODE SEGMENT

ASSUME CS: CODE, DS: DATA

START: MOV AX, DATA

MOV DS, AX

MOV BX, OFFSET TABLE

MOV SI, 06H

A: JMP DWORD PTR[BX+SI+2]

.....

程序执行完标号为A的指令后: CS= (1200H) IP= (2300H)

TABLE +8 / +10

00
22
00
12

段间跳转
段间跳转转移
段间跳转转移
段间跳转转移
段间跳转转移
段间跳转转移
段间跳转转移
段间跳转转移
段间跳转转移
段间跳转转移

BX说明在
数据段

TABLE +8

示例4:

当程序执行到STOP时:

SI= (9) = 8 * 1

DI= ()

CX= (3)

FLAG= (0)

ZF= (0)

DATA SEGMENT

STR1 DB 'HELLO WORLD!'

STR2 DB 'HELLO WOOLD!'

COUNT DW 12 000CH

FLAG DB ?

DATA ENDS

CODE SEGMENT

ASSUME CS: CODE, DS: DATA, ES: DATA

START: MOV AX, DATA

MOV DS, AX

MOV ES, AX

LEA BX, FLAG

LEA SI, STR1

LEA DI, STR2

MOV CX, COUNT

CLD

REPE CMPSB

JZ NEXT1

MOV Byte PTR[BX], 00H

JMP STOP

NEXT1: MOV Byte PTR[BX], 0FFH

STOP:

可重入

段寄存器的初始

① 叫相等, 直至CX=0
② 存在不等且CX≠0

示例5：

如果从38011端口输入的是
45H，则程序执行完后：

BX= (8)

AL= ()

SUM0—SUM7的内容为：

FF, 0, FF, 0, 0, 0, FF, 0

AL 0100 0101 → 1
(45H)

10100010 → 0

```
DATA SEGMENT
    SUM DB 8 DUP (0)
DATA ENDS
CODE SEGMENT
    ASSUME CS: CODE, DS: DATA
START: MOV AX, DATA
        MOV DS, AX
        LEA BX, SUM
        MOV DX, 380H
        IN AL, DX
        MOV CX, 8
NEXT:  ROR AL, 1
        JNC NEXT1
        MOV Byte PTR[BX], 0FFH
        JMP NEXT2
NEXT1: MOV Byte PTR[BX], 0
NEXT2: INC BX
        LOOP NEXT
```

不含CF的循环右移1位

MOV Byte PTR[BX], 0FFH

MOV Byte PTR[BX], 0

INC BX

例：

- 定义20B的字符输入缓冲区BUFF以及20B的字节变量DATA；
- 显示字符串输入提示信息，然后从键盘接收字符 'Hello, my friends! '；
- 将BUFF中的字符串按从左到右的方向传送到DATA中。

DSEG SEGMENT

MESS DB 'Please input string:', 0DH, 0AH, '\$'

BUFF DB 20, ?, 20 DUP(?)

DATA DB 20 Dup(?)

DSEG ENDS

CSEG SEGMENT

ASSUME CS : CSEG, DS : DSEG, ES : DSEG

START : MOV AX, DSEG

MOV DS, AX

MOV ES, AX

AGAIN : LEA DX, MESS

MOV AH, 9

INT 21H

回车换行

字符串输入缓冲区

普通缓冲区

初始化

显示字符串

输入 { LEA DX, BUFF
MOV AH, 10
INT 21H

XOR CX, CX 清0

MOV CL, BUFF+1

LEA DI, DATA

LEA SI, BUFF+2

CLD

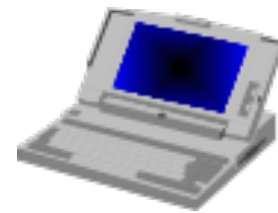
REP MOVSB

MOV AH, 4CH

INT 21H

CSEG ENDS

END SATRT



本章注意点

- 完整的汇编语言源程序结构
 - 定义逻辑段，说明段的含义，初始化段寄存器
- 伪指令
 - 数据定义方式
- 字符及字符串的输入和显示输出
 - 字符输入缓冲区的定义，输出字符串的定义
- 源程序的编写
 - 几种结构（顺序、循环、分枝等）